Deadlocks in Operating Systems

Prof. Mehdi Pirahandeh

Contents

1 Introduction to Deadlocks

In operating systems, a **deadlock** is a situation where a set of threads or processes are blocked indefinitely, waiting for resources held by each other. Understanding deadlocks is essential in designing systems that can manage resources efficiently and prevent resource conflicts.

1.1 System Model

The system model for deadlocks includes:

- **Resource Types (R1, R2, ..., Rm)**: Resources like CPU cycles, memory, and I/O devices.
- **Resource Instances**: Each resource type R_i has W_i instances.
- **Resource Usage by Processes**: Processes request, use, and release resources in this order.

2 Deadlock Characterization

A deadlock occurs if four conditions hold simultaneously:

1. **Mutual Exclusion**: Only one process can use a resource at a time.
2. **Hold and Wait**: A process holding resources can request additional ones.
3. **No Preemption**: Resources cannot be forcibly taken from a process.
4. **Circular Wait**: A set of processes are waiting on each other in a circular chain.

Condition	**Description**
Mutual Exclusion	Only one thread at a time can use a resource
Hold and Wait	A thread holding at least one resource is waiting for others
No Preemption	Resources cannot be forcibly released by the OS
Circular Wait	A circular chain of threads, each waiting for a resource held by the next

Table 1: Conditions Required for Deadlock

3 Resource-Allocation Graph

The resource-allocation graph is a tool to represent the allocation of resources to processes. It includes:

- **Threads (T1, T2, ..., Tn)**: Represented as nodes.

- **Resource Types (R1, R2, ..., Rm)**: Also represented as nodes.
- **Edges**: Request edges (Ti → Rj) indicate a thread waiting for a resource; assignment edges (Rj → Ti) show a resource allocated to a thread.

3.1 Basic Facts about Deadlocks in Graphs

- If there are no cycles in the graph, there is no deadlock.
- If a cycle exists, deadlock is possible if only one instance per resource type exists.

4 Methods for Handling Deadlocks

Several strategies are used to manage deadlocks in systems:

1. **Deadlock Prevention**: Ensures that the system never enters a deadlock state by invalidating one of the four deadlock conditions.
2. **Deadlock Avoidance**: Requires additional information to make decisions that avoid unsafe states.
3. **Deadlock Detection and Recovery**: Allows deadlocks to occur but detects and resolves them afterward.

5 Deadlock Prevention

Deadlock prevention aims to invalidate one of the four conditions required for deadlock.

5.1 Mutual Exclusion

Mutual exclusion is generally necessary for non-sharable resources. However, resources like read-only files can be shared.

5.2 Hold and Wait

Processes can be required to request all resources at once or release held resources before requesting new ones. This method reduces resource utilization and can lead to starvation.

```
void requestResources() {
    wait(mutex);
    if (holdsOtherResources) {
        releaseAllResources();
    }
    requestAllResources();
    signal(mutex);
}
```

Listing 1: Example of Hold and Wait Prevention

5.3 No Preemption

If a process holding resources requests another that is unavailable, all held resources are released and reassigned when available.

5.4 Circular Wait

Ordering resources by assigning a unique number to each and requiring resources to be requested in ascending order can prevent circular wait.

6 Deadlock Avoidance

Deadlock avoidance ensures that the system never enters an unsafe state. The system requires information about each process's maximum potential needs.

6.1 Banker's Algorithm

The Banker's algorithm dynamically checks resource allocation to ensure that the system remains in a safe state.

- **Available**: Vector indicating the number of available instances of each resource.
- **Max**: Maximum demand of each process for each resource.
- **Allocation**: Current allocation of resources to each process.
- **Need**: Remaining resource needs for each process.

Term	**Definition**
Available	Vector showing available instances of each resource type
Max	Matrix showing maximum demands of each process
Allocation	Matrix showing resources currently allocated
Need	Matrix showing resources still required by processes

Table 2: Banker's Algorithm Data Structures

7 Deadlock Detection and Recovery

If the system allows deadlocks, it must include mechanisms to detect and recover from them.

7.1 Detection Algorithm

A cycle-detection algorithm can identify deadlocks by analyzing the wait-for graph. The complexity of the algorithm is $O(n^2)$.

7.2 Recovery from Deadlock

Recovery methods include process termination and resource preemption. Processes can be aborted one by one until deadlock is eliminated, or resources can be preemptively reclaimed.

```
void preemptResource(thread T) {
    rollback(T);
    reassignResources();
}
```

Listing 2: Example of Deadlock Recovery by Resource Preemption

8 Examples of Deadlock Concepts

This section provides practical examples for each concept covered in the lecture on deadlocks. These examples are designed to help students understand how deadlock conditions manifest in real systems and how each prevention and detection method can be applied.

8.1 Example for System Model

Consider an operating system with two processes, P_1 and P_2, and two resource types, R_1 and R_2, each with one instance. Process P_1 requests R_1 and P_2 requests R_2. If P_1 subsequently requests R_2 while holding R_1, and P_2 requests R_1 while holding R_2, a deadlock will occur due to mutual resource holding.

8.2 Example for Deadlock Characterization

Imagine a printer and a scanner being shared between two processes, P_A and P_B.

- **Mutual Exclusion**: Only one process can use the printer or scanner at a time.
- **Hold and Wait**: P_A holds the printer and waits for the scanner, while P_B holds the scanner and waits for the printer.
- **No Preemption**: The printer and scanner cannot be forcibly taken away from P_A or P_B.
- **Circular Wait**: P_A is waiting for P_B's scanner, and P_B is waiting for P_A's printer, creating a circular dependency.

This scenario satisfies all conditions for deadlock.

8.3 Example for Resource-Allocation Graph

Consider a system with two processes, P_1 and P_2, and two resources, R_1 and R_2. The resource-allocation graph would have:

- Nodes for P_1, P_2, R_1, and R_2.
- A request edge from P_1 to R_1 and from P_2 to R_2.
- An assignment edge from R_1 to P_2 and from R_2 to P_1.

In this case, there is a cycle, indicating a potential deadlock.

8.4 Example for Deadlock Prevention

8.4.1 Mutual Exclusion Example

For a file that needs read-only access by multiple processes, mutual exclusion can be relaxed by allowing shared access instead of exclusive access.

8.4.2 Hold and Wait Example

In a memory management system, a process is required to release all held memory before requesting new memory blocks. This prevents the hold-and-wait condition, as the process is not allowed to hold resources while waiting for others.

8.4.3 No Preemption Example

In a CPU scheduling scenario, if a process holding the CPU requests additional memory that is currently unavailable, the system will release the CPU from this process and assign it to another process. This prevents deadlock by preempting resources in certain conditions.

8.4.4 Circular Wait Example

To prevent circular wait, a system enforces an ordering on resources. For example, if there are three resources R_1, R_2, and R_3, processes must request resources in a specific order (e.g., R_1 first, then R_2, and finally R_3).

8.5 Example for Deadlock Avoidance

8.5.1 Banker's Algorithm Example

Assume a system with three resources (A, B, C) and three processes (P1, P2, P3) with the following maximum resource demands and current allocations: Using the Banker's

Process	Max (A, B, C)	Allocation (A, B, C)	Need (A, B, C)
P1	(7, 5, 3)	(0, 1, 0)	(7, 4, 3)
P2	(3, 2, 2)	(2, 0, 0)	(1, 2, 2)
P3	(9, 0, 2)	(3, 0, 2)	(6, 0, 0)

Table 3: Example Setup for Banker's Algorithm

algorithm, the system checks whether it can allocate resources without entering an unsafe state by evaluating each request against the available resources.

8.6 Example for Deadlock Detection and Recovery

8.6.1 Detection Algorithm Example

Consider three processes, P_1, P_2, and P_3, each waiting for resources held by another in a cycle. The detection algorithm identifies the cycle in the wait-for graph, recognizing the deadlock.

8.6.2 Recovery from Deadlock Example

If a deadlock is detected, the system can choose to terminate one of the processes. For example, terminating P_1 may release resources, breaking the deadlock and allowing P_2 and P_3 to proceed.

Examples are critical to understanding deadlock conditions, prevention techniques, and detection mechanisms. These examples illustrate how theoretical concepts apply in real-world scenarios, making it easier to identify and resolve deadlocks in operating systems.

9 Conclusion

Understanding deadlocks and managing them is critical in operating systems to ensure efficient and reliable resource utilization. Methods such as deadlock prevention, avoidance, detection, and recovery provide various strategies for handling deadlocks in complex systems.